

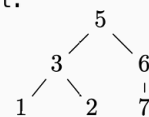
week_10

1. 看一个有意思的题

Definition. A pile (of leaves) for a tree t with no repeated leaf labels is a dictionary in which the label for each leaf of t is a key, and its value is the path from that leaf to the root. Each path from a node to the root is either an empty tuple, if the node is the root, or a two-element tuple containing the label of the node's parent and the rest of the path (i.e., the path to the root from the node's parent).

```
def pile(t):  
    """Return a dict that contains every path from a leaf to the root of tree t.
```

```
>>> pile(Tree(5, [Tree(3, [Tree(1), Tree(2)]), Tree(6, [Tree(7)])]))  
{1: (3, (5, ())), 2: (3, (5, ())), 7: (6, (5, ()))}
```



```
    p = {}
```

```
    def gather(u, path):  
        if u.is_leaf():
```

```
            p[u.label] = path
```

Base case:
Put a leaf label in p

```
        for b in u.branches:
```

```
            gather(b, (u.label, path))
```

Recursive call:
Build a longer path

Start at
the top

```
    gather(t, ())
```

all paths
have ()

```
    return p
```

u has a label that can
be added to the path

2. 再看一个有意思的题

Implement `sums`, which takes two positive integers n and k . It returns a list of lists containing all the ways that a list of k positive integers can sum to n (in any order).

```
def sums(n, k):  
    """Return the ways in which K positive integers can sum to N.
```

```
>>> sums(2, 2)
```

```
[[1, 1]]
```

```
>>> sums(2, 3)
```

```
[]
```

```
>>> sums(4, 2)
```

```
[[3, 1], [2, 2], [1, 3]]
```

```
>>> sums(5, 3)
```

```
[[3, 1, 1], [2, 2, 1], [1, 3, 1], [2, 1, 2], [1, 2, 2], [1, 1, 3]]
```

```
    """
```

```
    if k == 1:
```

```
        return [[n]]
```

```
    y = []
```

```
    for x in range(1, n):
```

Choose the first
number

```
        y.extend([s + [x] for s in sums(n - x, k - 1)])
```

```
    return y
```

Build the list

Choose a way

Ways of completing the list

3. 看一下这个题，思路非常重要。首先，该树是多叉树，所以必须使用 `for b in branches` 这样的代码；此外，自己想判断是否有子树，`for b in branches` 也同样可以解决这个问题；最后，由于是从底层开始改变代码，因此在这里只能使用递归，并且是先访问，再进行改变。

Write a function `cumulative_mul` that mutates the Tree `t` so that each node's label is replaced by the product of its label and the labels of all its descendants.

Hint: Be careful of the order in which you mutate the current node's label and process its subtrees; which one should come first?

```
def cumulative_mul(t):
    """Mutates t so that each node's label becomes the product of its own
    label and all labels in the corresponding subtree rooted at t.

    >>> t = Tree(1, [Tree(3, [Tree(5)]), Tree(7)])
    >>> cumulative_mul(t)
    >>> t
    Tree(105, [Tree(15, [Tree(5)]), Tree(7)])
    >>> otherTree = Tree(2, [Tree(1, [Tree(3), Tree(4), Tree(5)]), Tree(6, [Tree(7)])])
    >>> cumulative_mul(otherTree)
    >>> otherTree
    Tree(5040, [Tree(60, [Tree(3), Tree(4), Tree(5)]), Tree(42, [Tree(7)])])
    """
    for b in t.branches:
        cumulative_mul(b)
    total = t.label
    for b in t.branches:
        total *= b.label
    t.label = total

# Alternate solution using only one loop
def cumulative_mul(t):
    for b in t.branches:
        cumulative_mul(b)
    t.label *= b.label
```

4. 论 max 函数的巧用

Removing some nodes from a tree is called *pruning* the tree.

Complete the function `prune_small` that takes in a Tree `t` and a number `n`. For each node with more than `n` branches, keep only the `n` branches with the smallest labels and remove (*prune*) the rest.

Hint: The `max` function takes in an `iterable` as well as an optional `key` argument (which takes in a one-argument function). For example, `max([-7, 2, -1], key=abs)` would return `-7` since `abs(-7)` is greater than `abs(2)` and `abs(-1)`.

```
def prune_small(t, n):
    """Prune the tree mutatively, keeping only the n branches
    of each node with the smallest labels.

    >>> t1 = Tree(6)
    >>> prune_small(t1, 2)
    >>> t1
    Tree(6)
    >>> t2 = Tree(6, [Tree(3), Tree(4)])
    >>> prune_small(t2, 1)
    >>> t2
    Tree(6, [Tree(3)])
    >>> t3 = Tree(6, [Tree(1), Tree(3, [Tree(1), Tree(2), Tree(3)]), Tree(5, [Tree(3), Tree(4)])])
    >>> prune_small(t3, 2)
    >>> t3
    Tree(6, [Tree(1), Tree(3, [Tree(1), Tree(2)])])
    """
    while len(t.branches) > n:
        largest = max(t.branches, key=lambda x: x.label)
        t.branches.remove(largest)
    for b in t.branches:
        prune_small(b, n)
```

5. 这个题的设计也是巧妙用extend， 并且是先访问，再处理

Implement `delete`, which takes a Tree `t` and removes all non-root nodes labeled `x`. The parent of each remaining node is its nearest ancestor that was not removed. The root node is never removed, even if its label is `x`.

```
def delete(t, x):
    """Remove all nodes labeled x below the root within Tree t. When a non-leaf
    node is deleted, the deleted node's children become children of its parent.

    The root node will never be removed.

    >>> t = Tree(3, [Tree(2, [Tree(2), Tree(2)]), Tree(2), Tree(2, [Tree(2, [Tree(2), Tree(2)])])]
    >>> delete(t, 2)
    >>> t
    Tree(3)
    >>> t = Tree(1, [Tree(2, [Tree(4, [Tree(2)]), Tree(5)]), Tree(3, [Tree(6), Tree(2)]), Tree(4)])
    >>> delete(t, 2)
    >>> t
    Tree(1, [Tree(4), Tree(5), Tree(3, [Tree(6)]), Tree(4)])
    >>> t = Tree(1, [Tree(2, [Tree(4), Tree(5)]), Tree(3, [Tree(6), Tree(2)]), Tree(2, [Tree(6), Tree(2), Tree(7), Tree(8)]), Tree(4)])
    >>> delete(t, 2)
    >>> t
    Tree(1, [Tree(4), Tree(5), Tree(3, [Tree(6)]), Tree(6), Tree(7), Tree(8), Tree(4)])
    """
    new_branches = []
    for b in t.branches:
        delete(b, x)
        if b.label == x:
            new_branches.extend(b.branches)
        else:
            new_branches.append(b)
    t.branches = new_branches
```